Reviewer Pack — Burau Trace Defect of Clause Braid Lower-Bounds Tree-Resolut…

Entry b455093f5476 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 18:27:50 UTC

Burau Trace Defect of Clause Braid Lower-Bounds Tree-Resolution Steps

Entry ID: `b455093f5476` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-17 18:27:50 UTC

1. Conjecture

Field A (mathematical branch): Braid-group representation theory — the unreduced Burau representation ρ_t: B_n → GL_n(Z[t,t^{-1}]) specialised at t = −1, where Burau factors through the integer symplectic group Sp_{2g}(Z) (Burau 1936; Squier 1984; Birman 1974), and the trace defect ω(β) := |n − tr(ρ_{-1}(β))|. This corner of knot/braid theory has essentially no presence in proof complexity (arXiv search ‘Burau representation’ AND (‘Frege’ OR ‘resolution refutation’ OR ‘proof complexity’) returns no direct hits; <5 adjacent papers). **Field B** (complexity object): Tree-like Resolution / DPLL refutation step count for unsatisfiable 3-CNFs, viewed as a Frege-depth proxy in the Cook–Reckhow–Krajíček bounded-arithmetic framework where ¬F is Σ^b_0 and log_2 of the canonical tree size controls V^0-witnessing complexity (Beame–Pitassi 2001; Ben-Sasson–Wigderson 2001).

Statement:

> For an unsat 3-CNF F on n variables with m clauses, construct a braid word β(F) ∈ B_n by reading clauses C ∈ F in lex order: for each C with literals l_1, l_2, l_3 sorted by |var|, append generators σ_{|l_1|}^{sgn(l_1)·sgn(l_2)} · σ_{|l_2|}^{sgn(l_2)·sgn(l_3)} (σ_k for exponent +1, σ_k^{-1} for exponent −1, with σ_k acting between strands k and k+1, k ≤ n−1; pairs that straddle strand n−1 are skipped). Let M(F) = ρ_{-1}(β(F)) be the unreduced Burau matrix product specialised at t = −1 (so each generator is an integer n × n matrix with entries in {−1, 0, 1, 2}), and ω(F) := |n − tr(M(F))|. Conjecture: there is an absolute constant C ≥ 1/4 such that the canonical lex-DPLL refutation backtrack count t(F) satisfies 4·(t(F)^2 + 1) ≥ ω(F) for every unsat 3-CNF F. A single unsat F with 4·(t(F)^2 + 1) < ω(F) falsifies the conjecture.

Rationale (proposer’s reasoning):

> At t = −1 the Burau representation lands inside Sp_{2g}(Z), so the matrix product realises clause-pair conflicts as integer symplectic transvections and the trace defect quantifies how entangled the resulting symplectic flow becomes — a relational, non-commutative invariant of the SYNTACTIC clause ordering that is invisible to width or expansion alone. Any tree-Resolution refutation must split on every variable participating in an irreducible conflict, so a highly entangled clause braid should force backtracking. The invariant is structural (CNF syntax, not truth-table), sidestepping NATURAL_PROOFS; braids are non-abelian and do not lift to polynomial extensions of the Boolean ring, sidestepping ALGEBRIZATION; the construction uses no diagonalisation, sidestepping RELATIVIZATION.

Taxonomy category: `PROOF_COMPLEXITY_EXT_FREGE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c6ebba77ed512264`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across ~480 DPLL-verified unsat 3-CNFs (30 seeds per n∈{8,10,12,14,16}, α=6), compute ω(F) and t(F) per spec. Conjecture is SUPPORTED iff every accepted instance satisfies 4·(t(F)^2+1) ≥ ω(F) (support fraction = 1.0); FALSIFIED iff any single accepted instance violates it.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.85	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Burau representation proof complexity resolution refutation` - `braid group SAT CNF DPLL lower bound` - `knot invariant Boolean satisfiability tree resolution`

Top relevant hits considered: - [s2:68cab25640a4debd147cd412a08597d352b3397a] Refutation of a Proof Complexity Approach to Maxsat Resolution-Based Systems - [s2:10.1145/3506702] On Proof Complexity of Resolution over Polynomial Calculus

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import sys
import random
import math
import json
from fractions import Fraction

def matrix_mult(a, b):
    n = len(a)
    result = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
```

```
                result[i][j] += a[i][k] * b[k][j]
    return result

def matrix_trace(m):
    n = len(m)
    trace = 0
    for i in range(n):
        trace += m[i][i]
    return trace

def burau_generator(n, k, exponent):
    matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        if i == k:
            matrix[i][i] = 1
        elif i == k + 1:
            matrix[i][i] = -1
            matrix[i][k] = exponent
        else:
            matrix[i][i] = 1
    return matrix

def generate_3cnf(n, m, seed):
    random.seed(seed)
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause_vars = random.sample(variables, 3)
        clause = []
        for var in clause_vars:
            if random.random() < 0.5:
                clause.append(-var)
            else:
                clause.append(var)
        clauses.append(clause)
    return clauses

def is_unsat(clauses, n):
    assignments = [None] * (n + 1)
    for clause in clauses:
        for lit in clause:
            var = abs(lit)
            if assignments[var] is None:
                assignments[var] = lit > 0
            elif (assignments[var] and lit < 0) or (not assignments[var] and lit > 0):
                return False
    return True

def lex_dpll(clauses, n):
    assignments = [None] * (n + 1)
    backtracks = 0

    def backtrack(level):
        nonlocal backtracks
        if level == n + 1:
            return True
```

```
        var = level
        for val in [True, False]:
            assignments[var] = val
            if is_satisfiable(clauses, assignments):
                if backtrack(level + 1):
                    return True
            assignments[var] = None
            backtracks += 1
        return False

    def is_satisfiable(clauses, assignments):
        for clause in clauses:
            satisfied = False
            for lit in clause:
                var = abs(lit)
                if assignments[var] is not None:
                    if (assignments[var] and lit > 0) or (not assignments[var] and lit < 0):
                        satisfied = True
                        break
            if not satisfied:
                return False
        return True

    backtrack(1)
    return backtracks

def run_trial(seed):
    n_values = [8, 10, 12, 14, 16]
    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        m = 6 * n
        clauses = generate_3cnf(n, m, seed)
        if not is_unsat(clauses, n):
            continue

        instances_tested += 1
        beta = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        for clause in sorted(clauses):
            l1, l2, l3 = sorted(clause, key=lambda x: abs(x))
            k1 = abs(l1) - 1
            k2 = abs(l2) - 1
            if k1 >= n - 1 or k2 >= n - 1:
                continue
            exponent1 = (1 if l1 > 0 else -1) * (1 if l2 > 0 else -1)
            exponent2 = (1 if l2 > 0 else -1) * (1 if l3 > 0 else -1)
            g1 = burau_generator(n, k1, exponent1)
            g2 = burau_generator(n, k2, exponent2)
            beta = matrix_mult(beta, matrix_mult(g1, g2))

        omega = abs(n - matrix_trace(beta))
        t = lex_dpll(clauses, n)
```

```
        if 4 * (t ** 2 + 1) < omega:
            conjecture_holds = False
            counterexample = f"n={n}, m={m}, omega={omega}, t={t}"

        metric_values.append(omega)

    if not metric_values:
        return {
            "metric_name": "omega",
            "metric_value": 0,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No valid instances generated"
        }

    metric_value = sum(metric_values) / len(metric_values)

    return {
        "metric_name": "omega",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []
    metric_values = []
    conjecture_holds_all = True
    first_counterexample = ""

    for seed in seeds:
        result = run_trial(seed)
        results.append(result)
        metric_values.append(result["metric_value"])
        if not result["conjecture_holds"]:
            conjecture_holds_all = False
            if not first_counterexample:
                first_counterexample = result["counterexample"]
        print(f"TRIAL: {json.dumps({'seed': seed, **result})}")

    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if not conjecture_holds_all:
        print(f'RESULT: FALSIFIED counterexample="{first_counterexample}" first_failing_seed={seed
    elif support_fraction >= 0.8:
        print(f'RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    else:
        print('RESULT: INCONCLUSIVE reason=insufficient_support')
```

6. Per-seed results

Seed	Metric value	Holds?	Counterexample
11	0	✗	No valid instances generated
23	0	✗	No valid instances generated
37	0	✗	No valid instances generated
53	0	✗	No valid instances generated
71	0	✗	No valid instances generated
89	0	✗	No valid instances generated
103	0	✗	No valid instances generated
127	0	✗	No valid instances generated
149	0	✗	No valid instances generated
167	0	✗	No valid instances generated
191	0	✗	No valid instances generated
211	0	✗	No valid instances generated
233	0	✗	No valid instances generated
257	0	✗	No valid instances generated
277	0	✗	No valid instances generated
311	0	✗	No valid instances generated
347	0	✗	No valid instances generated
389	0	✗	No valid instances generated
421	0	✗	No valid instances generated
463	0	✗	No valid instances generated
503	0	✗	No valid instances generated
547	0	✗	No valid instances generated
593	0	✗	No valid instances generated
631	0	✗	No valid instances generated
677	0	✗	No valid instances generated
727	0	✗	No valid instances generated
773	0	✗	No valid instances generated
821	0	✗	No valid instances generated
877	0	✗	No valid instances generated
929	0	✗	No valid instances generated

Aggregate statistics:

Statistic	Value
n_seeds	30
metric_mean	0.0
metric_std	0.0
metric_ci95_half	0.0
metric_min	0
metric_max	0
support_fraction	0.0

7. Test stdout (last 2KB)


```
alue": 0, "instances_tested": 0, "conjecture_holds": false, "counterexample": "No valid instances gen
TRIAL: {"seed": 463, "metric_name": "omega", "metric_value": 0, "instances_tested": 0, "conjecture_ho
TRIAL: {"seed": 503, "metric_name": "omega", "metric_value": 0, "instances_tested": 0, "conjecture_ho
TRIAL: {"seed": 547, "metric_name": "omega", "metric_value": 0, "instances_tested": 0, "conjecture_ho
TRIAL: {"seed": 593, "metric_name": "omega", "metric_value": 0, "instances_tested": 0, "conjecture_ho
TRIAL: {"seed": 631, "metric_name": "omega", "metric_value": 0, "instances_tested": 0, "conjecture_ho
TRIAL: {"seed": 677, "metric_name": "omega", "metric_value": 0, "instances_tested": 0, "conjecture_ho
TRIAL: {"seed": 727, "metric_name": "omega", "metric_value": 0, "instances_tested": 0, "conjecture_ho
TRIAL: {"seed": 773, "metric_name": "omega", "metric_value": 0, "instances_tested": 0, "conjecture_ho
```

```
TRIAL: {"seed": 821, "metric_name": "omega", "metric_value": 0, "instances_tested": 0, "conjecture_ho
TRIAL: {"seed": 877, "metric_name": "omega", "metric_value": 0, "instances_tested": 0, "conjecture_ho
TRIAL: {"seed": 929, "metric_name": "omega", "metric_value": 0, "instances_tested": 0, "conjecture_ho
RESULT: FALSIFIED counterexample="No valid instances generated" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The empirical test is vacuous: every one of the 30 seeds reports 'No valid instances generated' with instances_tested=0 and metric=0. The conjecture was never actually evaluated on any unsat 3-CNF, so there is zero empirical support — this is a construction gap / instance-generator failure, not evidence. Additionally, support_fraction is 0.0, meaning even the harness reports the conjecture as not holding on the (empty) sample.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> All 30 seeds produced zero accepted instances ('No valid instances generated'), so the conjecture was never actually evaluated; the RESULT line's 'FALSIFIED' label reflects a generator failure rather than a genuine counterexample. Pre-registered acceptance threshold (n_accepted ≥ 400) was not remotely met. | next: Fix the instance generator (e.g., loosen acceptance, verify DPLL unsat check, ensure clauses produce non-skipped Burau generators at α=6, n∈{8..16}) and rerun to obtain a non-empty sam

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	344410
2	propose	claude_max	opus	0	0	355329
3	preregistration	claude_max	opus	0	0	6519
4	novelty	claude_max	opus	0	0	3704
5	novelty	claude_max	opus	0	0	7042
6	test_gen	mistral	codestral-latest	0	0	314181
7	test_gen	mistral	codestral-latest	0	0	325036
8	test_gen	mistral	codestral-latest	0	0	324466
9	test_gen	mistral	codestral-latest	0	0	315239
10	critic	claude_max	opus	0	0	8707
11	judge	claude_max	opus	0	0	6369

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 2011001 ms total latency. Provider mix: {'claude_max': 7, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/b455093f5476.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b455093f5476.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b455093f5476.tar.gz` (if generated)